

# Error Handling in Reflection

Document #: P3560R1 [[Latest](#)] [[Status](#)]  
Date: 2025-05-19  
Project: Programming Language C++  
Audience: EWG, LEWG  
Reply-to: Peter Dimov  
<[pdimov@gmail.com](mailto:pdimov@gmail.com)>  
Barry Revzin  
<[barry.revzin@gmail.com](mailto:barry.revzin@gmail.com)>

## Contents

---

- [1 Revision History](#)
- [2 Introduction](#)
- [3 Exception Type](#)
  - [3.1 Encoding](#)
  - [3.2 Single or Multiple Types](#)
  - [3.3 Derivation from `std::exception`](#)
- [4 Recoverable or Unrecoverable](#)
- [5 Proposed Wording](#)
  - [5.1 \[`meta.reflection.synop`\]](#)
  - [5.2 \[`meta.reflection.exception`\]](#)
  - [5.3 \[`meta.reflection.operators`\]](#)
  - [5.4 \[`meta.reflection.names`\]](#)
  - [5.5 \[`meta.reflection.queries`\]](#)
  - [5.6 \[`meta.reflection.member.queries`\]](#)
  - [5.7 \[`meta.reflection.layout`\]](#)
  - [5.8 \[`meta.reflection.extract`\]](#)
  - [5.9 \[`meta.reflection.substitute`\]](#)
  - [5.10 \[`meta.reflection.result`\]](#)
  - [5.11 \[`meta.reflection.define.aggregate`\]](#)
  - [5.12 \[`meta.reflection.traits`\]](#)
  - [5.13 Feature-Test Macro](#)
- [6 References](#)

## § 1 Revision History

---

Since [\[P3560R0\]](#):

- after discussion in an SG16 telecon, LEWG discussion, and [\[P3637R0\] \(Inherit `std::meta::exception` from `std::exception`\)](#), we changed the proposed `std::meta::exception` type to inherit from

`std::exception` ([see below](#)), and thus have a `char const* what()` accessor in addition to the `u8string_view u8what()` one.

— rebased on [\[P2996R12\]](#)

## § 2 Introduction

---

In [\[P2996R12\]\\_\(Reflection for C++26\)](#), we had to answer the question of what the error handling mechanism should be. We considered four options:

1. Returning an invalid reflection (similar to NaN for floating point)
2. Returning a `std::expected<T, E>` for some reflection-specific error type E
3. Failing to be a constant expression
4. Throwing an exception of type E, for some type E.

Option (1) doesn't work well, because not all reflection functions return `std::meta::info`. Some (such as `members_of`) return `vector<info>`, some (such as `identifier_of`) return `string_view`, and `extract<T>` even returns T. A NaN reflection doesn't solve the problem.

Option (2) places a heavy syntactic burden on user code, because `std::expected` needs to be unwrapped manually, without help from the language.

Option (3) doesn't provide any means for user code to recover from an error.

At the time we had to make the decision, option (4) was essentially equivalent to (3), because throwing an exception wasn't a constant expression, so we settled on option (3). However, since the adoption of [\[P3068R6\]\\_\(Allowing exception throwing in constant-evaluation\)](#), that has changed, and option (4) has become viable.

Using exceptions to signal errors doesn't suffer from the problem with option (1), because it's a strategy that can be used regardless of the return type. It also doesn't require syntactic changes to the user code.

Ordinarily, for runtime functions, exception handling might be avoided for reasons of binary size and runtime overhead; it also imposes the requirement that the API can't be used with exceptions disabled (which is nonstandard, but nevertheless highly popular.)

However, none of these objections apply to exceptions used at compile time. They have no binary footprint, don't affect the run time, and there is no reason for a compiler to not allow them even in “no exceptions” mode (because they are entirely contained to program compilation.)

Therefore, we believe that we need to adopt option (4) as the error handling strategy for reflection functions.

## § 3 Exception Type

---

To signal errors via throwing an exception, we need to settle on an exception type (or types) which to throw.

Since these exceptions will never escape to runtime, we don't need to be concerned with deriving their type(s) from `std::exception`. However, it would be desirable for the exceptions to carry enough information for error recovery (when caught), enough information for high quality error messages (when uncaught), and for them to be suitable for error handling in user `constexpr` and `constexpr` functions as well, in addition to standard ones.

To that end, we proposed the following exception type:

```
namespace std::meta {  
  
class exception  
{  
public:  
    constexpr exception(u8string_view what,  
                        info from,                                source_location where =  
                        source_location::current());  
  
    constexpr u8string_view what() const;  
    constexpr info from() const;  
    constexpr source_location where() const;  
};  
  
}
```

`exception::what()` is a string describing the error; `exception::from()` is a reflection of the function (or function template) from a call to which the error originated; and `exception::where()` is the source location of the call to that function.

For example, the following function

```
constexpr auto f()  
{  
    return members_of(^int);  
}
```

will throw an exception of type `std::meta::exception` for which `what()` will return (for example) `u8"invalid reflection operand"`, `from()` will return `^std::meta::members_of`, and `where()` will return a `std::source_location` object pointing at the call to `members_of` inside `f`.

Suppose a user wishes to write a `constexpr` function that only accepts class type reflections. It would be possible to use `std::meta::exception` to signal errors as follows:

```

constexpr auto user_fn(info type, source_location where =
    source_location::current())
{
    if( !is_class_type(type) )
    {
        throw std::meta::exception(u8"not a class type", ^^user_fn,
            where);
    }

    // carry on
}

```

## § 3.1 Encoding

What encoding should we use for the string describing the error, and what character type?

The encoding is left unspecified in the runtime case (`std::exception::what()`), which is generally regarded as a defect ([\[LWG4087\]](#)). Since we are designing a new component, we should not repeat that mistake, and specify the encoding of `meta::exception::what()`.

Since the string describing the error can be constructed from components coming from multiple sources, it should use an encoding that can represent any of these substrings. That is, it should use UTF-8.

The principled way to reflect this fact in the type system is to use `u8string_view`. However, there are strong, purely pragmatic, arguments in favor of using `string_view` instead.

`char8_t` has nearly zero support in the standard library, which makes it *very* inconvenient to use. Suppose, for example, that we are writing a function `member_of(info x, size_t i)` that returns `members_of(x)[i]`:

```

constexpr info member_of(info x, size_t i, source_location where =
    source_location::current())
{
    auto v = members_of(x);

    if( i >= v.size() )
    {
        throw meta::exception( u8"invalid member index", ^^member_of,
            where );
    }

    return v[i];
}

```

Further suppose that we want to provide a more descriptive error string, e.g. `"152 is not a valid member index"`, where 152 is the value of `i`.

There's basically no way to easily do that today. We can't use `std::format` to create a `u8string`, there is no `std::to_u8string`, there is even no equivalent of `to_chars` that would produce a `char8_t` sequence.

In contrast, if the constructor took `std::string_view`, we could have used any of these.

So maybe we should just use `string_view`? But that's not consistent with the current state of [\[P2996R12\]](#). It did start out using `string_view` everywhere, and had to be rewritten to supply additional `u8string_view` interfaces, for good reasons.

Consider, for instance, `std::meta::identifier_of(x)`. It can fail for two reasons: if the entity to which `x` refers has no associated identifier, or if it does, but that identifier is not representable in the literal encoding.

We are changing these failures from hard errors (not a constant expression) to throwing `meta::exception`. A sketch implementation of `identifier_of`, then, would look like this:

```
constexpr string_view identifier_of(info x)
{
    if( !has_identifier(x) )
    {
        throw meta::exception(u8"entity has no identifier",
                               ^^identifier_of, ...);
    }

    auto id = u8identifier_of(x);

    if( !is_representable(id) )
    {
        throw meta::exception(u8"identifier '"s + id + u8"'is not
                               representable", ^^identifier_of, ...);
    }

    // convert id to the literal encoding and return it
}
```

For quality of implementation reasons, we want to include the identifier in the error description string we pass to the exception constructor, so that the subsequent error message will say `"the identifier 'риба' is not representable"` and not just `"identifier not representable"`.

There is no way to do that if we take and return `string_view` from the exception constructor and `what()`. Since the failure is caused by the identifier not being representable in the literal encoding, it trivially follows that we can't put it into an error string that uses the literal encoding.

That is why we believe that taking and returning `u8string_view` is essential in order to maintain consistency with the current design of [\[P2996R12\]](#), which is the result of extensive discussions in SG16.

To address the usability question, after the SG16 telecon on February 5th, 2025, we decided to provide a dual API, like the rest of [\[P2996R12\]](#), and have two constructors, one taking `u8string_view` and one taking `string_view`:

```
namespace std::meta {
```

```

class exception
{
private:
    u8string what_;           // exposition only
    info from_;              // exposition only
    source_location where_;   // exposition only

public:
    constexpr exception(u8string_view what, info from,
        source_location where = source_location::current()) noexcept
        : what_(what), from_(from), where_(where) {}

    constexpr exception(string_view what, info from,
        source_location where = source_location::current()) noexcept
        : what_(ordinary-to-u8(what)), from_(from), where_(where) {}

    constexpr u8string_view u8what() const noexcept {
        return what_;
    }

    constexpr string what() const noexcept {
        return u8-to-ordinary(what_);
    }

    // ...
};

```

where `what()` fails to be constant if it cannot transcode. It would be nice if we had at least *u8-to-ordinary* and *ordinary-to-u8* already specified and present but, well, today is better than tomorrow.

This gives us a maximally usable API — since the standard library has plenty of support for string formatting and that can be used here, the conversion from ordinary to UTF-8 is fine. It does still mean that attempting to call `what()` could fail, but... so be it.

## § 3.2 Single or Multiple Types

We are proposing a single exception type. The runtime analogy is `std::system_error` as opposed to a hierarchy of exception types.

This in principle makes user code that wishes to inspect the failure reason and do different things depending on it less convenient to write. It would have to look like this

```

catch( meta::exception const& x )
{
    if( x.from() == ^^identifier_of )
    {
        // handle errors originating from identifier_of
    }
    else if( x.from() == ^^members_of )

```

```

    {
        // handle errors originating from members_of
    }
    // ...
}

```

instead of, hypothetically, like this

```

catch( meta::identifier_exception const& x )
{
    // handle errors originating from identifier_of
}
catch( meta::members_exception const& x )
{
    // handle errors originating from members_of
}
// ...

```

(exception type names are illustrative.)

We don't propose an exception hierarchy here. Designing a proper exception hierarchy is not something we can realistically do in the C++26 timeframe. It's not as straightforward as just using an exception per function because functions can fail for multiple reasons, and client code may well wish to distinguish between these.

Furthermore, an exception hierarchy can be designed at a later date, with the functions changed to throw an appropriate type derived from the currently proposed `meta::exception`. Code written against this proposal will continue to work unmodified, and new code would be able to use more specific catch clauses.

## § 3.3 Derivation from `std::exception`

Our initial proposal did not derive `std::meta::exception` from `std::exception`, because `meta::exception` only exists at compile time, whereas `std::exception`-derived exceptions inhabit the runtime domain.

However, we repeatedly received suggestions to the contrary, and we now think that the derivation would be desirable for consistency with all other standard exceptions, some of which will end up being used at compile time as well.

`std::exception::what` returns `char const*`, rather than `std::string` as in the interface listed above, but this is surmountable by a slight modification. We just need to keep an optional `<string>` member with the what string in the ordinary encoding, in addition to the `u8string` member that holds the what string in UTF-8. If the conversion from UTF-8 to ordinary fails, we leave the optional disengaged and in that case `what()` fails.

```

namespace std::meta {

class exception
{
private:
    u8string u8what_;           // exposition only
    optional<string> what_;     // exposition only
    info from_ ;               // exposition only
    source_location where_;     // exposition only

public:
    constexpr exception(u8string_view what, info from,
        source_location where = source_location::current()) noexcept
        : u8what_(what), from_(from), where_(where) {
        if(u8-to-ordinary-would-succeed(what))
            what_ = u8-to-ordinary(what);
    }

    constexpr exception(string_view what, info from,
        source_location where = source_location::current()) noexcept
        : u8what_(ordinary-to-u8(what)), what_(what), from_(from),
          where_(where) {}

    constexpr u8string_view u8what() const noexcept {
        return u8what_;
    }

    constexpr char const* what() const noexcept {
        return what_->c_str();
    }

    // ...
};

}

```

## § 4 Recoverable or Unrecoverable

We went through the proposed API in [\[P2996R12\]](#) and we think that all of the library functions should be recoverable — that is failing to meet the requirements of the function should be an exception rather than constant evaluation failure — with a single exception, `std::meta::define_aggregate`.

`define_aggregate` isn't likely to be used from a context from which recovery is meaningful, and even if it were, for meaningful recovery we would have to guarantee that the partial effects of a failure have been rolled back (as a definition containing some of the members may already have been produced at the point where the error is detected.) We don't believe that imposing this requirement is warranted or worth the cost.

The rest of the library functions are straightforwardly fallible, so the ability to recover from them is desirable.



## § 5 Proposed Wording

The wording here introduces a new type `std::meta::exception` and defines it.

Otherwise it's pretty rote changing all the error handling from something of the form “*Constant When: C*” to “*Throws: meta::exception unless C*”.

### § 5.1 [meta.reflection.synop]

Add to the synopsis in [meta.reflection.synop:]

```
namespace std::meta {
    using info = decltype(^^::);

+ // [meta.reflection.exception], class exception
+ class exception;

    // ...
}
```

### § 5.2 [meta.reflection.exception]

Add a new subclause as follows:

**Class `exception`, [meta.reflection.exception]**

```
class exception : std::exception
{
private:
    optional<string> what_;    // exposition only
    u8string u8what_;         // exposition only
    info from_;              // exposition only
    source_location where_;   // exposition only

public:
    constexpr exception(u8string_view what, info from,
        source_location where = source_location::current()) noexcept;

    constexpr exception(string_view what, info from,
        source_location where = source_location::current()) noexcept;

    exception(exception const&) = default;
    exception(exception&&) = default;

    exception& operator=(exception const&) = default;
    exception& operator=(exception&&) = default;

    constexpr const char* what() const noexcept override;
    constexpr u8string_view u8what() const noexcept;
```

```

    consteval info from() const noexcept;
    consteval source_location where() const noexcept;
};

```

1 Reflection functions throw exceptions of type `std::meta::exception` to signal an error. `std::meta::exception` is a `constexpr`-only type.

```

    consteval exception(u8string_view what, info from,
        source_location where = source_location::current()) noexcept;

```

2 *Effects:* Initializes `u8what_` with `what`, `from_` with `from` and `where_` with `where`. If `what_` can be represented in the ordinary literal encoding, initializes `what_` with `what`, transcoded from UTF-8 to the ordinary literal encoding.

```

    consteval exception(string_view what, info from,
        source_location where = source_location::current()) noexcept;

```

3 *Effects:* Initializes `what_` with `what`, `u8what_` with `what` transcoded from the ordinary literal encoding to UTF-8, `from_` with `from` and `where_` with `where`.

```

    constexpr const char* what() const noexcept override;

```

4 *Constant When:* `what_.has_value()` is true.

5 *Returns:* `what_ -> c_str()`.

```

    consteval u8string_view u8what() const noexcept;

```

6 *Returns:* `what_`.

```

    consteval info from() const noexcept;

```

7 *Returns:* `from_`.

```

    consteval source_location where() const noexcept;

```

8 *Returns:* `where_`.

## § 5.3 [meta.reflection.operators]

Replace the error handling in this subclause:

```

    consteval operators operator_of(info r);

```

2 ~~Constant When~~ *Throws:* `meta::exception` unless `r` represents an operator function or operator function template.

3 *Returns:* The value of the enumerator from `operators` whose corresponding *operator-function-id* is the unqualified name of the entity represented by `r`.

```
constexpr string_view symbol_of(operators op);
constexpr u8string_view u8symbol_of(operators op);
```

4 ~~Constant When~~ Throws: `meta::exception unless` The value of `op` corresponds to one of the enumerators in `operators`.

5 Returns: `string_view` or `u8string_view` containing the characters of the operator symbol name corresponding to `op`, respectively encoded with the ordinary literal encoding or with UTF-8.

## § 5.4 [meta.reflection.names]

Replace the error handling in this subclause:

```
constexpr string_view identifier_of(info r);
constexpr u8string_view u8identifier_of(info r);
```

1 Let  $E$  be UTF-8 if returning a `u8string_view`, and otherwise the ordinary literal encoding.

2 ~~Constant When~~ Throws: `meta::exception unless` `has_identifier(r)` is `true` and the identifier that would be returned (see below) is representable by  $E$ .

## § 5.5 [meta.reflection.queries]

Replace the error handling in this subclause:

```
constexpr info type_of(info r);
```

1 ~~Constant When~~ Throws: `meta::exception unless` `has-type(r)` is `true`.

```
constexpr info object_of(info r);
```

2 ~~Constant When~~ Throws: `meta::exception unless`  $r$  is a reflection representing either

(2.1) — an object with static storage duration ([basic.stc.general]), or

(2.2) — a variable that either declares or refers to such an object, and if that variable is a reference  $R$  then either

(2.2.1) —  $R$  is usable in constant expressions ([expr.const]), or

(2.2.2) — the lifetime of  $R$  began within the core constant expression currently under evaluation.

```
constexpr info value_of(info r);
```

3 Let  $R$  be a constant expression of type `info` such that  $R == r$  is `true`.

4 ~~Constant When~~ **Throws**: `meta::exception unless`  $[ : R : ]$  is a valid *splice-expression* (`[expr.prim.splice]`).

```
constexpr info parent_of(info r);
```

5 ~~Constant When~~ **Throws**: `meta::exception unless` `has_parent(r)` is `true`.

```
constexpr info template_of(info r);  
constexpr vector<info> template_arguments_of(info r);
```

6 ~~Constant When~~ **Throws**: `meta::exception unless` `has_template_arguments(r)` is `true`.

## § 5.6 [meta.reflection.member.queries]

Replace the error handling in this subclause:

```
constexpr vector<info> members_of(info r, access_context ctx);
```

1 ~~Constant When~~ **Throws**: `meta::exception unless`  $r$  is a reflection representing either a class type that is complete from some point in the evaluation context or a namespace.

```
constexpr vector<info> bases_of(info type, access_context ctx);
```

2 ~~Constant When~~ **Throws**: `meta::exception unless` `dealias(type)` is a reflection representing a complete class type.

```
constexpr vector<info> static_data_members_of(info type, access_context  
ctx);
```

3 ~~Constant When~~ **Throws**: `meta::exception unless` `dealias(type)` represents a complete class type.

```
constexpr vector<info> nonstatic_data_members_of(info type,  
access_context ctx);
```

- 4 ~~Constant When~~ Throws: `meta::exception unless` `dealias(type)` represents a complete class type.

```
constexpr vector<info> enumerators_of(info type_enum);
```

- 5 ~~Constant When~~ Throws: `meta::exception unless` `dealias(type_enum)` represents an enumeration type and `is_enumerable_type(type_enum)` is `true`.

## § 5.7 [meta.reflection.layout]

Replace the error handling in this subclause:

```
constexpr member_offset offset_of(info r);
```

- 1 ~~Constant When~~ Throws: `meta::exception unless` `r` represents a non-static data member, unnamed bit-field, or direct base class relationship other than a virtual base class of an abstract class.

```
constexpr size_t size_of(info r);
```

- 2 ~~Constant When~~ Throws: `meta::exception unless` `dealias(r)` is a reflection of a type, object, value, variable of non-reference type, non-static data member that is not a bit-field, direct base class relationship, or data member description ( $T$ ,  $N$ ,  $A$ ,  $W$ ,  $NUA$ ) ([class.mem.general]) where  $W$  is not  $\perp$ . If `dealias(r)` represents a type, then `is_complete_type(r)` is `true`.

```
constexpr size_t alignment_of(info r);
```

- 3 ~~Constant When~~ Throws: `meta::exception unless` `dealias(r)` is a reflection representing a type, object, variable of non-reference type, non-static data member that is not a bit-field, direct base class relationship, or data member description. If `dealias(r)` represents a type, then `is_complete_type(r)` is `true`.

```
constexpr size_t bit_size_of(info r);
```

- 4 ~~Constant When~~ Throws: `meta::exception unless` `dealias(r)` is a reflection of a type, object, value, variable of non-reference type, non-static data member, unnamed bit-field, direct base class relationship, or data member description. If `dealias(r)` represents a type  $T$ , there is a point within the evaluation context from which  $T$  is not incomplete.

## § 5.8 [meta.reflection.extract]

Replace the error handling in this subclause:

```
template <class T>
    consteval T extract-ref(info r); // exposition only
```

1 [ Note 1: T is a reference type. — end note ]

2 ~~Constant When~~ Throws: meta::exception unless

(2.1) — r represents a variable or object of type U,

(2.2) — is\_convertible\_v<remove\_reference\_t<U>(\*)[], remove\_reference\_t<T>(\*)  
[]> is true, and [ Note 2: The intent is to allow only qualification conversions from  
U to T. — end note ]

(2.3) — if r represents a variable, then either that variable is usable in constant expressions or  
its lifetime began within the core constant expression currently under evaluation.

```
template <class T>
    consteval T extract-member-or-function(info r); // exposition only
```

3 ~~Constant When~~ Throws: meta::exception unless

(3.1) — r represents a non-static data member with type X, that is not a bit-field, that is a  
direct member of a class C and T is X C::\*;

(3.2) — r represents an implicit object member function with type F or F noexcept that is a  
direct member of a class C and T is F C::\* or

(3.3) — r represents a non-member function, static member function, or explicit object  
member function of function type F or F noexcept and T is F\*.

```
template <class T>
    consteval T extract-val(info r); // exposition only
```

4 Let U be the type of the value that r represents.

5 ~~Constant When~~ Throws: meta::exception unless

(5.1) — U is a pointer type, T and U are similar types ([conv.qual]), and  
is\_convertible\_v<U, T> is true,

(5.2) — U is not a pointer type and the cv-unqualified types of T and U are the same, or

(5.3) — U is a closure type, T is a function pointer type, and the value that r represents is  
convertible to T.

## § 5.9 [meta.reflection.substitute]

Replace the error handling in this subclause:

```
template <reflection_range R = initializer_list<info>>
constexpr bool can_substitute(info templ, R&& arguments);
```

- 1 ~~Constant When~~ **Throws:** `meta::exception unless` `templ` represents a template and every reflection in `arguments` represents a construct usable as a template argument (`[temp.arg]`).

```
template <reflection_range R = initializer_list<info>>
constexpr info substitute(info templ, R&& arguments);
```

- 2 ~~Constant When~~ **Throws:** `meta::exception unless` `can_substitute(templ, arguments)` is `true`.

## § 5.10 [meta.reflection.result]

Replace the error handling in this subclause:

- 1 An object *O* of type *T* is *meta-reflectable* if an lvalue expression denoting *O* is suitable for use as a constant template argument for a constant template parameter of type *T*& (`[temp.arg.nontype]`).

- 2 The following are defined for exposition only to aid in the specification of `reflect_value`:

```
template <typename T>
constexpr info reflect-value-scalar(T expr); // exposition only
```

Let *V* be the value computed by an lvalue-to-rvalue conversion applied to `expr`.

- 3 ~~Constant When~~ **Throws:** `meta::exception unless` *V* satisfies the constraints for the result of a prvalue constant expression (`[expr.const]`) and, if *V* is a pointer to an object, then that object is meta-reflectable.

- 4 **Returns:** A reflection of a value of type *T* associated with the computed value *V*.

```
template <typename T>
constexpr info reflect-value-class(T const& expr); // exposition only
```

- 5 **Mandates:** *T* is copy constructible and structural (`[temp.param]`).

- 6 Let *O* be an object copy-initialized from `expr`.

- 7 ~~Constant When~~ **Throws:** `meta::exception unless`

- (7.1) —  $0$  satisfies the constraints for the result of a glvalue constant expression ([expr.const]),
- (7.2) — every object referred to by a constituent reference of  $0$ , or pointed to by a constituent pointer value of  $0$ , is meta-reflectable.
- 8 *Returns:* A reflection of a value of type  $T$  associated with a template parameter object that is template-argument-equivalent to  $0$ .
- ```
template <typename T>
constexpr info reflect_value(const T& expr);
```
- \* *Effects:* [...]
- ```
template <typename T>
constexpr info reflect_object(T& expr);
```
- 9 *Mandates:*  $T$  is not a function type.
- 10 ~~Constant When~~ *Throws:* `meta::exception` unless `expr` designates a meta-reflectable object.
- 11 *Returns:* A reflection of the object designated by `expr`.

## § 5.11 [meta.reflection.define.aggregate]

Replace the error handling in this subclause:

- ```
constexpr info data_member_spec(info type,
                                data_member_options options);
```
- 1 ~~Constant When~~ *Throws:* `meta::exception` unless the following conditions are met:
- (1.1) — `dealias(type)` represents a type `cv T` where  $T$  is either an object type or a reference type;
- (1.2) — if `options.name` contains a value, then:
- (1.2.1) — `holds_alternative<u8string>(options.name->contents)` is `true` and `get<u8string>(options.name->contents)` contains a valid identifier when interpreted with UTF-8, or
- (1.2.2) — `holds_alternative<string>(options.name->contents)` is `true` and `get<string>(options.name->contents)` contains a valid identifier when interpreted with the ordinary literal encoding;
- (1.3) — otherwise, if `options.name` does not contain a value, then `options.bit_width` contains a value;
- (1.4) — if `options.alignment` contains a value, it is an alignment value ([basic.align]) not less than `alignment_of(type)`; and



- (1.5) — if `options.bit_width` contains a value  $V$ , then
  - (1.5.1) — `is_integral_type(type) || is_enumeration_type(type)` is `true`,
  - (1.5.2) — `options.alignment` does not contain a value,
  - (1.5.3) — `options.no_unique_address` is `false`, and
  - (1.5.4) — if  $V$  equals `0` then `options.name` does not contain a value.

## § 5.12 [meta.reflection.traits]

Replace the error handling for all the type traits:

- 1 Subclause [meta.reflection.traits] specifies consteval functions to query the properties of types ([meta.unary]), query the relationships between types ([meta.rel]), or transform types ([meta.trans]) at compile time. Each consteval function declared in this class has an associated class template declared elsewhere in this document.
- 2 Every function and function template declared in this clause ~~has the following conditions required for a call to that function or function template to be a constant subexpression~~ `((defs.const.subexpr))` throws an exception of type `std::meta::exception` unless the following conditions hold:
  - (2.1) — For every parameter  $p$  of type `info`, `is_type(p)` is `true`.
  - (2.2) — For every parameter  $r$  whose type is constrained on `reflection_range`, `ranges::all_of(r, is_type)` is `true`.
- 3 [...]

## § 5.13 Feature-Test Macro

Bump `__cpp_lib_reflection` in 17.3.2 [version.syn] (which isn't there yet) to some new value:

```
+ #define __cpp_lib_reflection 2025XXL // also in <meta>
```

# § 6 References

[LWG4087] Victor Zverovich. Standard exception messages have unspecified encoding.

<https://wg21.link/lwg4087>

[P2996R12] Barry Revzin, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevoorde, and Dan Katz. 2025-05-11. Reflection for C++26.

<https://wg21.link/p2996r12>

[P3068R6] Hana Dusíková. 2024-11-19. Allowing exception throwing in constant-evaluation.

<https://wg21.link/p3068r6>

[P3560R0] Barry Revzin, Peter Dimov. 2025-01-12. Error Handling in Reflection.

<https://wg21.link/p3560r0>

[P3637R0] Victor Zverovich, Nevin Liber, Michael Hava. 2025-03-08. Inherit std::meta::exception from std::exception.

<https://wg21.link/p3637r0>